

CST8326: Web Programming

Assignment 2: Dynamic Web Application (20%)

HTML, CSS, JavaScript ,Node.js Express.JS, and Mongoose DB

Objectives

- Allows you to use your creativity, to develop a dynamic web application. You will choose what you want to build, with the project requirements constraining the features that must be used.
- Have hands-on experience with several technologies and can make these components interact with each other to provide services.
- Work as a team, collaborate with your project partner(s) professionally and contribute fairly.

Read the entire instruction before starting. If at any time you are unsure or are having problems, consult your lab instructor.

Required Equipment: Collaboration Tool (GitHub recommended)

Part I: Choosing Your Team (Due: March 15)



This is a team project; Teams will consist of maximum **four students** When you have decided on your team configuration register your team on the Brightspace.

Part II: Choosing Your Project



For this project you will create a dynamic web application which meets the following minimum criteria: **Please consider the following as minimum requirements only, you are encouraged to get creative and do more than this, but this is what will be required to receive full marks.**

1. A **website or web application**: that will have user-based privileges so that the experience is personalized based on who is logged into the system. Users must log into the services to be able to see and interact with the whole of the website. You should also have some parts of the website open to the public, but at least a portion will be only accessible to logged in users.
2. **Backend Database Functionality**: Implement backend database functionality for seamless content management on the website. Users should be able to add, display, update and remove content through the website interface.
3. **Search and Filtering**: Allow users to search and filter content. The results should update dynamically by reloading the page after each search or filter action.
4. **HTML Structure**: Use semantically correct HTML to ensure proper document structure.
5. **User Registration Form**: Develop a user registration form allowing individuals to sign up as new users. Implement functionality to store user information in a database.
6. **Form Validation**: Implement client-side form validation using JavaScript and DOM manipulation. No standard HTML form validation is allowed; all validation will be done with JavaScript. Provide the user with appropriate error messages for a smooth user experience as discussed in the form validation lab.
7. **Dynamic Behavior**: Incorporate dynamic behavior with client-side input validation using JavaScript and DOM manipulation. Modify pages according to user inputs or the application's current states.
8. **Responsive Design**: An attractive, functional, and responsive layout including a distinct mobile and desktop mode without using any frameworks or libraries.
9. **Using templets**: Your application should use template rendering pug.

10. Adhere to good coding practices:

- Consistent Indentation:
- Meaningful Variable and Function Names
- Include comments for clarity.

11. Submitted code: The solution and code you submit must be unique. That is, it cannot be a copy of, or be too like, someone else's code, or other code found elsewhere. You are, however, free to use any code used in class code as part of your assignment solution. **[Assignment mark is 0 if this requirement is not met.]**

12. Technologies used: Your executable code must be written in JavaScript and must use the technologies and techniques taught in the course. We should be able to run your code using our Node.js installation by following the instructions provided in your report file. **[Assignment mark is 0 if this requirement is not met.]**

Possibilities for projects include

- a business website that allows custom registered user interactions.
- a cataloguing web application that allows a user to catalogue things like books, contacts, etc.
- content management application such as a simple database driven blogging tool.
- a basic social networking tool allowing users to connect and message each other.
- a restaurant app that manages orders taken from the public part of the webapp and is managed on the private part.

Remember, these are just examples to guide you. Feel free to choose a project idea that you're passionate about. If you have an idea that meets the requirements, go for it! This project is an opportunity for you to express your creativity and apply your skills in a way that excites you.

Part III: Project Submission



When the project is completed and tested you will package up the deliverables as follows:

- You must submit **all the necessary code** required to run your application, including:
 - Backend code (Node.js + Express)
 - Frontend code (HTML, CSS, and any client-side JavaScript)
 - Mongoose schema files (database models)
 - Any other supporting files (e.g., configuration files, package.json, etc.)

Your project must be runnable using our Node.js installation, following the instructions provided in your report file. [0 marks for the assignment if the required code is missing and the application cannot be run successfully].

- A report that contains the following:
 - A cover page includes the Title of your project and Name(s) of group members.
 - site map and a wireframe of your web application.
 - A breakdown of the tasks assigned to each team member.
 - INSTALL INSTRUCTIONS: Instructions on what actual commands to execute to install any external code modules needed to run your code. This will likely look like `npm install` or `npm install module_name`. It must be a legitimate command that your lab professor can copy-paste-and-execute. The lab professor will attempt to execute your commands from a command terminal open on the directory that contains your server file. [0 marks for the assignment if the install command is missing, incorrect, or incomplete.]
- All code written for the project in any language. Identify **who wrote what**

portions of the code.

- You must submit all the code files and data files necessary to compile and run your app. The code must at least compile without error and launch. The lab professor will execute your app by following the instructions you provide in your report. You must submit a single.zip formatted file to brightspace. (not .rar or .tar or whatever; only .zip format is accepted) [0 marks for assignment if the zip file is missing].
- Your client-side and server-side javascript should not be in two giant files. Break you client-side javascript into smaller manageable and readable files and include them individually with <script> tags in your html document. On the server-side use requires or imports appropriately to organize your code into managable size files.
- If your assignment uses external modules installed with npm, DON'T submit the node_modules directory with your code (it's potentially huge). Remove that directory and only submit the package.json and package-lock.json files. The lab professor will use these files to install the required modules using the instructions in your report.
- Your slides for presentaion.

Each team submits only one copy. Upload the zip file to group assignment on Brightspace.

Part IV: Final Project Demo (Full Team)

All partners must be present at the demo.

Presentations

Prepare a 3-5 slides that represent your project.

Each participant should be able to describe and demo the features of the website that they have worked on. The team leader will schedule this demo. **If any team members do not attend the demo, they will be graded as zero in Assignment 2.**

You will have to sign-up for a demonstration slot in order to do this. A sign-up page will be available for you to use. An announcement with details will be sent out when a demo sign-up sheet becomes available. Please only **sign up for one slot and do not remove the slot of another group**. Sign up early so that you get the slot you want.

- **The work will be graded zero if you do not demo it on time, even if uploaded.**
- A demo is limited to 15 minutes (5 minutes demo + 10 minutes Q&A).

Grading

Criteria	Points	Description
Backend Database Functionality (30 point)		
Backend Database	30	Successfully implements backend database functionality. Allows users to add, display, update and remove content through the website interface. Uses appropriate database management
Front End (20)		
Front End	20	Enables users to search and filter for content as required. Uses semantically correct HTML to ensure proper document structure. Properly utilizes HTML5 semantic elements. Develop a user registration form. Includes necessary fields for user registration. Implements client-side form validation using JavaScript and DOM manipulation. Provides user-appropriate error messages for a smooth user experience. Creates a responsive and user-friendly interface using HTML and CSS. Ensures a consistent and pleasant user experience across different devices
Back End (30)		
Back End	30	Written in Node and Express Defines multiple routes. Uses a route parameter Reads a JSON request body and returns results in JSON. Use async/await to handle database access Use HTML templet

Adherence to Good Coding Practices (5points)		
	5	Consistent Indentation: Meaningful Variable and Function Names: Include comments for clarity.
Adherence all the Submission requirements (15 points)		
	15	All the Requirement of Part III: Project Submission.
Total	100	

Suggested Weekly Schedule

- Week 9 – Form a team (max four members), decide on a project concept (be creative!). Design a user interface, site map wireframing.
- Week 10 – Client-side Design (HTML/CSS,)
- Week 11– Client-side Scripting (JavaScript)
- Week 12– Server-side Components (backend)

Due Date

- April 3rd (no extension)